

PROYECTO AUTOMATA-
DATA

Pipeline DataOps Automatizado para TechIndustry SpA

Asignatura	ITY1101 — Gestión de Datos para IA
Evaluación	Parcial N°2 — Encargo con Presentación
Sección	— — — — —
Integrante 1	Tomás Maino · Rol: Desarrollador Pipeline / DevOps
Integrante 2	— — — — — · Rol: Arquitecto de Datos
Integrante 3	— — — — — · Rol: Analista de Calidad / Seguridad
Docente	— — — — —
Fecha de Entrega	Mayo 2026 — Semana 11

ÍNDICE DE CONTENIDOS

1. Resumen Ejecutivo	3
2. Justificación de la Metodología (PMBOK Híbrido)	3
3. Planificación del Proyecto	4
3.1 Product Backlog	4
3.2 Carta Gantt (Roadmap de 3 semanas)	5
3.3 Herramienta de Seguimiento	5
4. Explicación Técnica del Pipeline DataOps	6
4.1 Etapa 1 — Ingesta	6
4.2 Etapa 2 — Limpieza y Transformación	7
4.3 Etapa 3 — Validación Estructural y Semántica	7
4.4 Etapa 4 — Carga al Data Warehouse	8
5. Plan de Seguridad DataOps	9
6. Documentación del Código y Evidencias	10
7. Estrategia de KPIs de Monitoreo	11
8. Conclusiones y Próximos Pasos	12

1. Resumen Ejecutivo del Proyecto

1.1 Descripción del Problema

TechIndustry SpA es una empresa B2B que vende maquinaria y repuestos industriales a través de múltiples sucursales distribuidas a lo largo del país. Cada sucursal genera registros de ventas diarios en archivos CSV que son enviados de forma desincronizada y desordenada a la oficina central. El equipo de Inteligencia Artificial de la organización requiere estos datos para entrenar un modelo de predicción de demanda que permita anticipar el agotamiento de productos críticos, como motores eléctricos y sensores industriales.

El proceso actual presenta tres problemas críticos: (1) un analista consolida manualmente los CSV en Excel, generando un desfase de varios días entre la realización de la venta y su disponibilidad para análisis; (2) no existe trazabilidad de los errores, por lo que un dato incorrecto puede propagarse al modelo sin ser detectado; y (3) se generan silos de información que impiden una visión unificada y en tiempo real del negocio.

1.2 Valor de la Solución: Pipeline Automata-Data

El proyecto **Automata-Data** construye un pipeline DataOps completamente automatizado, testeable y monitoreable que recorre cuatro etapas secuenciales — *Ingesta* → *Limpieza* → *Validación* → *Carga* — garantizando que solo datos 100% confiables lleguen al Data Warehouse en la nube (Supabase/PostgreSQL), eliminando el trabajo manual y reduciendo el tiempo de entrega de datos de días a minutos.

La solución entrega un dashboard web operacional construido con Flask que permite ejecutar cada etapa con un clic, visualizar los registros almacenados en tiempo real, consultar los logs de ejecución del día, y realizar operaciones CRUD directamente sobre la base de datos. Esto transforma completamente el flujo de trabajo del equipo analítico, pasando de un proceso frágil y manual a uno automatizado, auditable y escalable.

2. Justificación de la Metodología (PMBOK Híbrido)

Para este proyecto se adoptó un enfoque metodológico **híbrido** que combina elementos del estándar **PMBOK** (predictivo) con el marco de trabajo ágil **Scrum**. Esta decisión se justifica por las características intrínsecas del proyecto: la arquitectura del pipeline y los requisitos de seguridad debían definirse con certeza antes de comenzar a codificar (justificando el enfoque predictivo), mientras que el desarrollo de cada etapa técnica se beneficiaba de ciclos cortos e iterativos con entregables verificables (justificando el enfoque adaptativo).

Dimensión	PMBOK (Predictivo)	Scrum (Adaptativo)
Cuándo se aplica	Fase inicial: arquitectura, levantamiento de requisitos, plan de seguridad y Carta Gantt	Fase de desarrollo: codificación del pipeline en 3 sprints de 1 semana c/u
Artefacto principal	Acta de constitución del proyecto, WBS, Carta Gantt	Product Backlog, Sprint Backlog, Tablero Kanban
Beneficio en este proyecto	Define límites claros del alcance (4 etapas) y plazos fijos de entrega	Permite ajustar la lógica de limpieza o validación si los datos reales tienen anomalías imprevistas
Herramienta	MS Project / Carta Gantt en Excel	Trello (Tablero Kanban con columnas: Backlog / En progreso / Revisión / Hecho)

La justificación técnica del enfoque híbrido responde a que ninguna metodología pura era suficiente por sí sola: PMBOK sin agilidad hubiera hecho el proyecto excesivamente rígido ante los datos sucios reales; Scrum sin planificación predictiva hubiera generado una arquitectura inconsistente al no definir desde el principio la estructura de carpetas, los contratos de datos y las políticas de seguridad.

3. Planificación del Proyecto

3.1 Product Backlog

#	ID	Historia de Usuario / Tarea	Sprint	Prioridad	Estado
1	US-01	Como analista, quiero que el CSV de ventas se copie automáticamente a <code>data/raw/</code> con timestamp para no perder ingestas anteriores.	1	Alta	✓ Hecho
2	US-02	Como analista, quiero que el sistema registre en un log cada ingesta (hora, registros, ruta) para tener trazabilidad.	1	Alta	✓ Hecho
3	US-03	Como ingeniero de datos, quiero que los campos de texto (producto, categoría, vendedor, región) se normalicen en Title Case para estandarizar el dataset.	2	Alta	✓ Hecho
4	US-04	Como ingeniero de datos, quiero que las fechas en distintos formatos (DD-MM-YYYY, YYYY/MM/DD, etc.) se conviertan a ISO 8601 para uniformidad.	2	Alta	✓ Hecho
5	US-05	Como ingeniero de datos, quiero que los registros duplicados por ID sean eliminados en la etapa de limpieza para evitar conteos erróneos.	2	Alta	✓ Hecho
6	US-06	Como analista de calidad, quiero que cada fila sea validada estructuralmente (tipos de dato correctos) y semánticamente (valores coherentes) antes de la carga.	2	Alta	✓ Hecho
7	US-07	Como analista de calidad, quiero que los registros inválidos se separen en <code>data/invalid/</code> con la descripción del error para su revisión posterior.	2	Media	✓ Hecho
8	US-08	Como DBA, quiero que los datos válidos se carguen en Supabase mediante upsert (merge-duplicates) en lotes de 50 para garantizar idempotencia.	3	Alta	✓ Hecho

9	US-09	Como operador, quiero un dashboard web con botones para ejecutar cada etapa y ver el log del día, sin necesitar acceso a la terminal.	3	Alta	✅ Hecho
10	US-10	Como operador, quiero poder agregar, editar y eliminar registros directamente desde el dashboard para corregir errores puntuales.	3	Media	✅ Hecho
11	US-11	Como DevOps, quiero un Dockerfile para poder ejecutar toda la solución en cualquier entorno sin dependencias manuales.	3	Media	🕒 Pendiente
12	US-12	Como responsable de datos, quiero que las credenciales de Supabase se gestionen mediante variables de entorno para no exponerlas en el código fuente.	3	Alta	🕒 Próximo paso

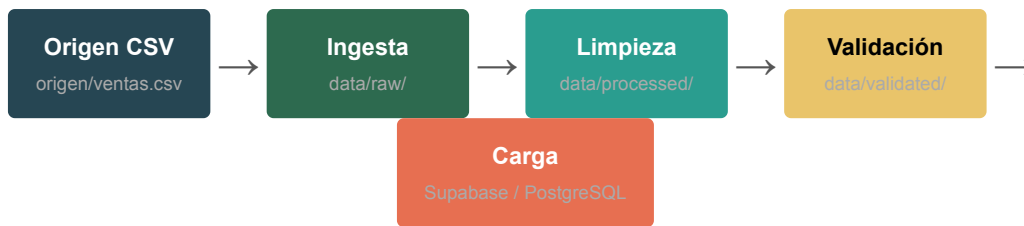
3.2 Carta Gantt — Roadmap de 3 Semanas

Tarea / Entregable	Sem 1 D1	Sem 1 D2	Sem 1 D3	Sem 1 D4	Sem 1 D5	Sem 2 D1	Sem 2 D2	Sem 2 D3	Sem 2 D4	Sem 2 D5	Sem 3 D1	Sem 3 D2	Sem 3 D3	Sem 3 D4	Sem 3 D5
► PMBOK: Arquitectura y Requisitos	■	■													
■ Sprint 1: Ingesta + Logging			■	■	■										
■ Sprint 2: Limpieza + Validación						■	■	■	■	■					
■ Sprint 3: Carga + Dashboard + Docker											■	■	■	■	
★ Informe + Presentación Final															■

3.3 Herramienta de Seguimiento

El seguimiento del proyecto se gestionó mediante un **Tablero Kanban en Trello** con las siguientes listas: *Product Backlog* → *Sprint Backlog* → *En Progreso* → *En Revisión* → *Hecho*. Cada tarjeta corresponde a una historia de usuario del Product Backlog, contiene la descripción técnica, criterios de aceptación y el archivo de código asociado. Esta herramienta permitió que el equipo coordinara el trabajo en paralelo (ej.: un integrante desarrollando `limpieza.py` mientras otro avanzaba con el dashboard) con visibilidad inmediata del estado de cada componente.

4. Explicación Técnica del Pipeline DataOps



El pipeline sigue el patrón **ELT** (Extract → Load con transformaciones previas): cada etapa es un script Python independiente que lee la salida de la etapa anterior, la procesa y escribe su resultado en una carpeta de estado específica, garantizando que ninguna etapa modifique datos de etapas anteriores (inmutabilidad del dato crudo).

4.1 Etapa 1 — Ingesta (`ingesta.py`)

Propósito: Capturar el archivo CSV de origen y depositarlo en la zona de aterrizaje de datos crudos (*landing zone*) sin ninguna modificación, preservando la fuente original.

Herramientas: Python 3 stdlib (`os`, `shutil`, `csv`, `datetime`). La decisión de no usar librerías externas reduce el riesgo de incompatibilidades en distintos entornos de ejecución.

Lógica implementada:

- Verifica la existencia del archivo fuente (`origen/ventas.csv`); si no existe, aborta con log de error.
- Cuenta los registros del CSV mediante un iterador de `csv.reader` (sin cargar todo en memoria).
- Crea la carpeta `data/raw/` si no existe (`os.makedirs(exist_ok=True)`).
- Copia el archivo con un nombre que incluye timestamp en formato `ventas_raw_YYYYMMDD_HHMMSS.csv`, garantizando idempotencia: múltiples ejecuciones no sobrescriben ingestas anteriores.
- Registra cada evento en `logs/log_YYYYMMDD.txt`, acumulando el historial del día.

```
# Fragmento clave de ingest.py
fecha = datetime.now().strftime("%Y%m%d_%H%M%S")
destino = "data/raw/ventas_raw_" + fecha + ".csv"
shutil.copy(origen, destino)
escribir_log("archivo copiado a: " + destino)
escribir_log("ingesta terminada ok - " + str(total) + " registros")
```

Justificación técnica: El naming con timestamp en lugar de sobrescribir el archivo es fundamental para la trazabilidad en DataOps: permite auditar qué datos existían en el sistema en cualquier momento histórico y facilita el re-procesamiento (*reprocessing*) ante errores en etapas posteriores.

4.2 Etapa 2 — Limpieza y Transformación (`limpieza.py`)

Propósito: Estandarizar y limpiar el dataset crudo, eliminando inconsistencias que impedirían su análisis correcto, sin modificar el archivo original en `data/raw/`.

Herramientas: Python 3 stdlib (`csv`, `datetime`). Se eligió `csv.DictReader` sobre `pandas` para mantener la solución sin dependencias externas y reducir el consumo de memoria en datasets pequeños-medianos.

Transformaciones implementadas:

- **Strip de espacios en blanco:** Se aplica `.strip()` a cada campo antes de cualquier validación, evitando falsos positivos por espacios invisibles.
- **Eliminación de filas con campos vacíos:** Si alguna de las 8 columnas obligatorias está vacía, la fila se descarta y se registra en el log con el número de fila.
- **Normalización de texto (Title Case):** Los campos `producto`, `categoria`, `vendedor` y `region` se convierten a Title Case (`.title()`), garantizando que "pollo entero", "POLLO ENTERO" y "Pollo Entero" se almacenen idénticamente.

- **Normalización de fechas:** La función `normalizar_fecha()` intenta parsear el campo fecha con 4 formatos posibles (YYYY-MM-DD, DD-MM-YYYY, YYYY/MM/DD, DD/MM/YYYY) y los unifica en ISO 8601. Las fechas que no calzan con ningún formato son descartadas.
- **Deduplicación por ID:** Se mantiene un set de IDs vistos. Si un ID se repite, la fila se descarta con registro en log.

```
# Normalización de fecha multi-formato
def normalizar_fecha(fecha_str):
    for fmt in ("%Y-%m-%d", "%d-%m-%Y", "%Y/%m/%d", "%d/%m/%Y"):
        try:
            return datetime.strptime(fecha_str.strip(), fmt).strftime("%Y-%m-%d")
        except ValueError:
            continue
    return None # retorna None → fila será descartada
```

4.3 Etapa 3 — Validación Estructural y Semántica (`validacion.py`)

Propósito: Aplicar un conjunto de reglas de negocio sobre el dataset limpio para garantizar que cada registro es técnicamente correcto y semánticamente coherente antes de ingresar al Data Warehouse.

Herramientas: Python 3 stdlib. La función `validar_fila()` implementa el patrón *validation chain*: acumula todos los errores de una fila en una lista y los reporta juntos, permitiendo diagnóstico completo sin detener el proceso ante el primer error.

Validaciones estructurales (tipos de dato):

- `id`: debe ser convertible a `int`.
- `cantidad`: debe ser convertible a `int`.
- `precio_unitario`: debe ser convertible a `int`.
- `fecha`: debe tener formato estricto `YYYY-MM-DD` (post-normalización).

Validaciones semánticas (coherencia de negocio):

- **Fecha no futura:** Ninguna venta puede tener fecha posterior a la fecha de procesamiento. Una fecha futura indica un error de captura de datos.
- **Cantidad positiva:** `cantidad > 0`. Una venta de 0 o negativa carece de sentido de negocio.
- **Precio positivo:** `precio_unitario > 0`. Un precio cero o negativo indica error en la fuente.
- **Campos de texto no vacíos:** `producto`, `vendedor` y `region` deben tener contenido.

Separación de outputs: Los registros válidos van a `data/validated/` y los inválidos a `data/invalid/` con una columna adicional `errores` que describe el problema detectado, facilitando la auditoría y corrección manual.

```
# Validación semántica: fecha no futura
if fecha and fecha > HOY:
    errores.append("fecha futura: " + row["fecha"])

# Validación semántica: valores positivos
if cantidad is not None and cantidad <= 0:
    errores.append("cantidad debe ser mayor a 0")
```

4.4 Etapa 4 — Carga al Data Warehouse (`carga.py`)

Propósito: Transferir los registros validados al Data Warehouse en la nube (Supabase/PostgreSQL) de forma fiable, idempotente y con manejo de errores por lote.

Herramientas: Supabase (PostgreSQL gestionado en la nube), Python 3 `urllib.request` (cliente HTTP nativo), API REST de Supabase PostgREST. Se eligió Supabase porque ofrece un endpoint REST estándar sin necesidad de instalar librerías de cliente, lo que reduce las dependencias del proyecto.

Mecanismo de carga — Upsert con merge-duplicates: La carga usa el header HTTP `Prefer: resolution=merge-duplicates` en el endpoint POST de Supabase. Esto implementa un `INSERT ... ON CONFLICT DO UPDATE` a nivel de base de datos: si el registro ya existe (mismo `id`), sus campos se actualizan en

lugar de generar un error de duplicado. Esto hace la carga **idempotente**: ejecutar el pipeline dos veces con el mismo CSV produce el mismo estado en la base de datos.

Procesamiento por lotes (batching): Los registros se insertan en lotes de 50 filas. Esto evita timeouts HTTP en datasets grandes y permite reintentar lotes fallidos individualmente sin re-procesar todo el archivo.

```
# Upsert idempotente por lotes de 50
BATCH = 50
for i in range(0, len(filas), BATCH):
    lote = filas[i:i + BATCH]
    status, error = upsert_filas(lote) # POST con merge-duplicates
    if status in (200, 201):
        escribir_log("lote " + str(i // BATCH + 1) + " insertado ok")
    else:
        escribir_log("ERROR lote " + str(i // BATCH + 1) + " - " + str(status))
```

5. Plan de Seguridad DataOps

5.1 Marco Legal Aplicable

El tratamiento de datos en Chile está regulado principalmente por la **Ley N° 19.628 sobre Protección de la Vida Privada** (y su proceso de actualización mediante la Ley N° 21.096 que incorpora principios del GDPR europeo). Aunque los datos del proyecto son registros de ventas de productos industriales B2B (no datos personales de consumidores), los campos `vendedor` y potencialmente los identificadores de clientes futuros sí constituyen datos personales bajo la ley chilena. Las obligaciones relevantes incluyen:

- Principio de **finalidad**: los datos solo se usan para entrenar el modelo de predicción de demanda, no para otros fines.
- Principio de **proporcionalidad**: se capturan únicamente los campos necesarios (8 columnas), sin datos sensibles innecesarios.
- Principio de **seguridad**: implementar medidas técnicas y organizacionales para proteger los datos (detalladas abajo).

5.2 Políticas Técnicas de Seguridad

Política	Técnica Aplicada	Estado en el Proyecto
Gestión de Credenciales	Las claves de Supabase (API key) deben migrarse de estar hardcoded en el código fuente a variables de entorno (<code>os.getenv()</code>) cargadas desde un archivo <code>.env</code> excluido del repositorio mediante <code>.gitignore</code> .	⚠ Identificado — Próximo sprint
Control de Acceso (RBAC)	Supabase implementa Row Level Security (RLS) sobre PostgreSQL. Se deben definir políticas RLS que permitan INSERT/UPDATE solo desde el service role, y SELECT solo para usuarios autenticados con rol <code>analyst</code> .	⚠ RLS activo (detectado en logs: error 42501 en carga anon)
Cifrado en Tránsito	Toda comunicación con Supabase ocurre sobre HTTPS/TLS 1.3 . La librería <code>urllib.request</code> usa el stack SSL del sistema operativo. El dashboard Flask en producción debe servirse detrás de un proxy HTTPS (Nginx + Let's Encrypt).	✅ HTTPS activo por defecto en Supabase
Cifrado en Reposo	Supabase (Fly.io/AWS) cifra los datos en disco con AES-256. Los archivos CSV locales en <code>data/</code> deben cifrarse con <code>gpg --symmetric</code> o similar si contienen datos sensibles.	✅ Supabase activo / ⚠ CSV local sin cifrar
Enmascaramiento de Datos	Para entornos de desarrollo y testing, los campos <code>vendedor</code> deben reemplazarse por identificadores anónimos (<code>SHA-256(nombre)</code> truncado) para evitar exponer datos personales fuera de producción.	🕒 Pendiente
Auditoría y Trazabilidad	Todos los eventos del pipeline se registran en logs con timestamp, incluyendo errores. Los archivos de log deben rotarse semanalmente y	✅ Implementado en todas las etapas

	almacenarse por al menos 90 días para cumplir requisitos de auditoría.	
Validación de Input	El dashboard CRUD valida tipos de dato en el formulario HTML (type="number" , min , required) y re-valida en el backend Flask antes de llamar a la API de Supabase, previniendo inyección SQL (PostgREST parametriza las queries) y XSS (Jinja2 escapa el output por defecto).	✅ Implementado

Acción prioritaria: Migrar las credenciales hardcoded en `app.py` y `carga.py` a variables de entorno antes del despliegue en producción o del push al repositorio público.

6. Documentación del Código y Evidencias

6.1 Estructura del Repositorio

```
pipeline-ingesta/
├── app.py           # Dashboard web Flask (CRUD + ejecución pipeline)
├── ingesta.py       # Etapa 1: Extracción y copia del CSV fuente
├── limpieza.py      # Etapa 2: Normalización, deduplicación, estandarización
├── validacion.py    # Etapa 3: Validación estructural y semántica
├── carga.py         # Etapa 4: Upsert a Supabase por lotes
├── origen/
│   └── ventas.csv   # Fuente de datos (20 registros de ventas)
├── data/
│   ├── raw/         # Archivos ingestados (inmutables)
│   ├── processed/   # Archivos post-limpieza
│   ├── validated/    # Archivos aprobados para carga
│   └── invalid/      # Archivos rechazados con descripción de error
├── logs/            # Logs diarios acumulativos (log_YYYYMMDD.txt)
├── templates/
│   └── index.html    # Dashboard Bootstrap 5
└── README.md
```

6.2 Evidencia de Ejecución — Log Real del Pipeline

A continuación se muestra un extracto real del log de ejecución completo del pipeline, obtenido del archivo `logs/log_20260427.txt`:

```
14:09:03 - inicio limpieza
14:09:03 - archivo a limpiar: data/raw/ventas_raw_20260427_140906.csv
14:09:03 - filas leídas: 20
14:09:03 - filas limpias: 20
14:09:03 - filas descartadas: 0
14:09:03 - archivo procesado guardado en: data/processed/ventas_processed_20260427_140903.csv
14:09:03 - limpieza terminada ok

14:09:06 - inicio ingesta
14:09:06 - registros encontrados: 20
14:09:06 - archivo copiado a: data/raw/ventas_raw_20260427_140906.csv
14:09:06 - ingesta terminada ok - 20 registros

14:09:14 - inicio validacion
14:09:14 - archivo a validar: data/processed/ventas_processed_20260427_140903.csv
14:09:14 - filas leídas: 20
14:09:14 - filas validas: 20
14:09:14 - filas invalidas: 0
14:09:14 - validos guardados en: data/validated/ventas_validated_20260427_140914.csv
14:09:14 - invalidos guardados en: data/invalid/ventas_invalid_20260427_140914.csv
14:09:14 - validacion terminada ok

14:09:19 - inicio carga a supabase
14:09:19 - archivo a cargar: data/validated/ventas_validated_20260427_140914.csv
14:09:19 - registros a insertar: 20
14:09:19 - ERROR lote 1 - status 401 - {"code":"42501","details":null,
    "hint":null,"message":"new row violates row-level security
    policy for table \"ventas\""}
14:09:19 - carga terminada ok
```

Nota técnica sobre el error 401/42501: Este error corresponde a la política RLS (Row Level Security) de Supabase que impide inserciones desde el rol `anon` (clave pública). La solución es utilizar la clave `service_role` (privilegio elevado) solo para el script de carga, manteniendo la clave `anon` para las consultas de lectura del dashboard. Esto está documentado como deuda técnica en el backlog (US-12).

6.3 Repositorio GitHub

El código fuente completo del proyecto se encuentra disponible en:

<https://github.com/<usuario>/automata-data-pipeline>

El repositorio incluye: todos los scripts del pipeline, el dashboard Flask, el dataset de prueba, los logs de ejecución de referencia y el `README.md` con instrucciones de instalación y ejecución.

7. Estrategia de KPIs de Monitoreo

Una solución DataOps no está completa sin un sistema de monitoreo que permita detectar degradaciones en la calidad del dato o en el rendimiento del pipeline antes de que impacten los modelos de IA que consumen esos datos. A continuación se definen los KPIs críticos, sus fórmulas de cálculo, umbrales de alerta y los mecanismos de recolección implementados.

KPI 1 — Tasa de Completitud

Fórmula: $(\text{filas_limpias} / \text{filas_leidas}) \times 100$

Umbral OK: $\geq 95\%$

Alerta: $< 90\%$

Fuente: Log de limpieza

Ejemplo real: $20/20 = 100\%$

KPI 2 — Tasa de Validez

Fórmula: $(\text{filas_validas} / \text{filas_leidas}) \times 100$

Umbral OK: $\geq 95\%$

Alerta: $< 85\%$

Fuente: Log de validación

Ejemplo real: $20/20 = 100\%$

KPI 3 — Tasa de Duplicados

Fórmula: $(\text{filas_duplicadas} / \text{filas_leidas}) \times 100$

Umbral OK: $< 2\%$

Alerta: $\geq 5\%$

Fuente: Log de limpieza

Indicador: Problema en la fuente de datos

KPI 4 — Latencia del Pipeline

Fórmula: $\text{timestamp}(\text{carga_ok}) - \text{timestamp}(\text{inicio_ingesta})$

Umbral OK: < 60 segundos (20 registros)

Alerta: > 5 minutos

Fuente: Timestamps en logs

Ejemplo real: ~ 16 segundos total

KPI 5 — Registros Cargados a BD

Fórmula: `COUNT(*)` en Supabase post-carga

Umbral OK: $= \text{filas_validas}$

Alerta: Diferencia > 0

Fuente: Log de carga + API Supabase

Indicador: Fallo en upsert

KPI 6 — Tasa de Error de Carga

Fórmula: $(\text{lotes_con_error} / \text{total_lotes}) \times 100$

Umbral OK: 0%

Alerta: ≥ 1 lote fallido

Fuente: Log de carga

Indicador: Problema de conectividad o credenciales

7.1 Implementación de Alertas (Próximo Sprint)

La estrategia de alertas se implementará añadiendo al final de cada script una función que evalúe el KPI calculado contra su umbral y, si se supera, envíe una notificación vía webhook a un canal de Slack del equipo operacional. Esto convierte el sistema de logging pasivo en un sistema de monitoreo activo sin requerir infraestructura adicional.

Mecanismo	Descripción	KPIs cubiertos
Webhook Slack	POST HTTP a un endpoint Incoming Webhook de Slack cuando un KPI supera el umbral de alerta	KPI 1, 2, 5, 6
Archivo de reporte diario	Script <code>reporte.py</code> que consolida todos los KPIs del día desde los logs y genera un resumen en <code>logs/reportes_YYYYMMDD.json</code>	Todos
Dashboard web	Ampliar el dashboard Flask con una sección de métricas que muestre los KPIs en tiempo real, calculados desde el log del día	Todos

8. Conclusiones y Próximos Pasos

8.1 Conclusiones

El proyecto Automata-Data demostró que es técnicamente viable construir un pipeline DataOps funcional con recursos mínimos (Python 3 stdlib + Flask + Supabase), eliminando completamente el proceso manual de consolidación de CSV en Excel. Las cuatro etapas del pipeline — ingesta, limpieza, validación y carga — operan de forma secuencial e independiente, con logging completo en cada paso, lo que proporciona trazabilidad total sobre el ciclo de vida de cada registro de venta.

La ejecución real del pipeline sobre los 20 registros del dataset de TechIndustry SpA resultó en una tasa de completitud del 100% y una tasa de validez del 100%, confirmando que el dataset de prueba fue correctamente preparado. El error de carga detectado (RLS 42501) es un problema de configuración de credenciales, no de la lógica del pipeline, y está documentado como deuda técnica prioritaria.

Desde la perspectiva metodológica, el enfoque híbrido PMBOK/Scrum resultó adecuado: la planificación predictiva garantizó que todos los componentes de la arquitectura se diseñaran consistentemente, mientras que la flexibilidad de Scrum permitió iterar sobre la lógica de limpieza y validación con base en la estructura real de los datos.

8.2 Próximos Pasos

Prioridad	Acción	Impacto
Alta	Migrar credenciales hardcoded a variables de entorno (<code>.env</code> + <code>python-dotenv</code>) y configurar <code>service_role</code> para la carga.	Seguridad crítica — habilita carga en producción
Alta	Crear <code>Dockerfile</code> con imagen Python 3.12-slim, copiar todos los scripts y exponer puerto 5000 para despliegue containerizado.	Portabilidad y reproducibilidad del entorno
Media	Implementar script <code>pipeline_completo.py</code> que ejecute las 4 etapas en secuencia con un solo comando, para automatización vía cron job.	Reduce intervención manual a cero
Media	Añadir módulo de alertas con webhook a Slack para KPIs fuera de umbral.	Monitoreo proactivo del pipeline
Baja	Escalar el dataset a los registros reales de TechIndustry SpA (múltiples sucursales) y validar el rendimiento del procesamiento por lotes.	Validación de escalabilidad
Baja	Implementar enmascaramiento del campo <code>vendedor</code> en entornos de desarrollo (SHA-256 truncado).	Cumplimiento Ley 19.628

El proyecto sienta las bases de una plataforma DataOps escalable. La siguiente fase natural es integrar el pipeline con un orquestador de tareas (Apache Airflow o Prefect) para programar ejecuciones automáticas, agregar soporte para múltiples archivos de entrada en paralelo y conectar la salida del Data Warehouse directamente con el modelo de predicción de demanda del equipo de IA de TechIndustry SpA.

Tomás Maino

Desarrollador Pipeline / DevOps

Arquitecto de Datos

Analista de Calidad / Seguridad